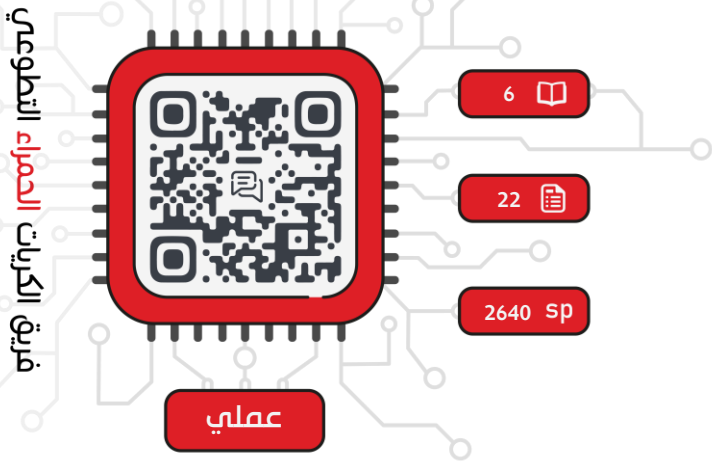


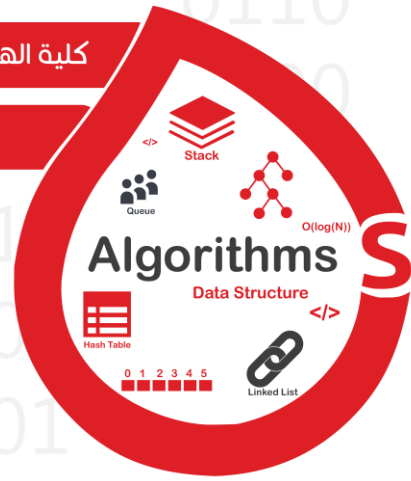


كلية الهندسة المعلوماتية

السنة الثانية

Stack & Queue

عملي مشترك



RB Informatics; 3/12/224

خوارزميات 1

السلام عليكم ورحمة الله وبركاته

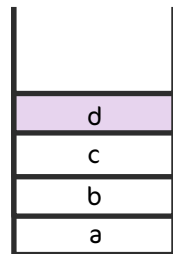
ماذا سنأخذ في محاضرة اليوم؟

- مفهوم الـ stack وتطبيقه (Stack concept and implementation)
- مفهوم الـ Queue وتطبيقه (Queue concept and implementation)
- تمارين 😊

Stack

مفهوم الـ stack (المكدس):

هو عبارة عن بنية معطيات خطية (linear data structure) يتبع مبدأ (LIFO) أو (Last-In-First-Out) والذي يعني أن آخر عنصر يتم إضافته هو أول عنصر يمكن الوصول إليه أو حذفه) وأول عنصر يتم إضافته هو آخر عنصر يمكن الوصول إليه (أو حذفه).



STACK

هذا العنصر الذي يمكننا الوصول إليه أو حذفه

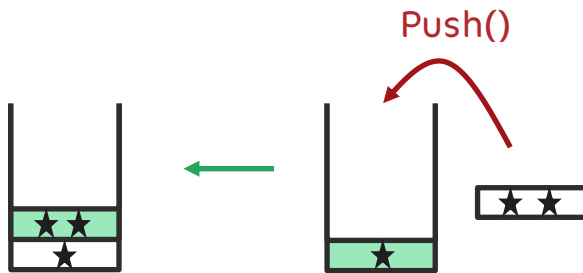
مثال:

يتم استخدام الـ stack عندما نقوم بـ undo للعمليات في أي برنامج (عندما نضغط Ctrl z).

العمليات على الـ stack:

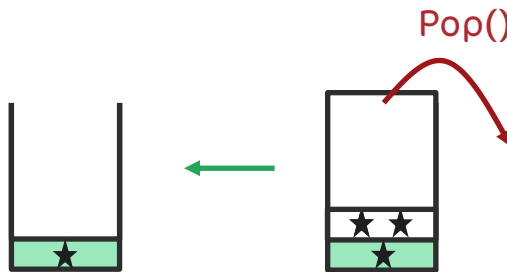
1. Push():

إضافة عنصر إلى الـ stack من الأعلى.



2. Pop():

حذف العنصر الموجود في أعلى الـ stack.

ملاحظة:

تابع الـ pop() في class الـ stack الجاهز الموجود في الـ java يقوم بإعادة قيمة العنصر الموجود أعلى الـ stack وحذفه.

3. Peek():

إعادة قيمة العنصر الموجود في أعلى الـ stack.



4. isEmpty():

يعيد (true) إذا كانت الـ stack فارغة، و (false) إذا كانت تحوي عنصر أو أكثر.

ملاحظة:

يجب التفريق بين المفهوم (concept) والتطبيق (implementation)، فالـ implementation هو تمثيل أو تطبيق المفهوم المجرد.

:Stack Implementation

لتطبيق أو تمثيل الـ stack نحتاج إلى:

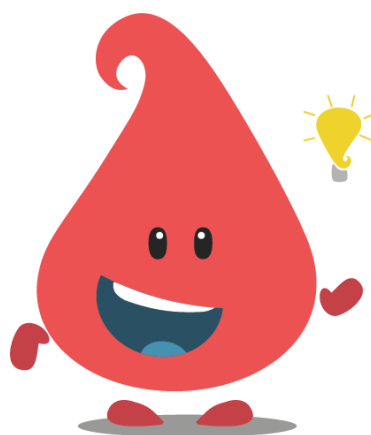
- Pointer واحد: يشير إلى آخر عنصر تمت إضافته.
- بنية معطيات خطية: Array أو LinkedList.

تمثيل الـ stack باستخدام الـ Array:

محتويات الـ class:

- **:Array**
وهي البنية التي تمثل الـ stack.
- **متغير من نوع int:**
يمثل index آخر عنصر تمت إضافته إلى الـ array (وليكن اسمه Top)
- **متغير من نوع int:**
يخزن حجم المصفوفة.
- **:Constructor**
نرسل له حجم الـ array والذي سيكون حجم الـ stack الخاصة بنا
- **:Methods**
isEmpty() - peek() - pop() - push()

```
public class MyStackA {  
    int[] arr;  
    int Top;  
    int size;  
    MyStackA(int size) {  
        this.size = size;  
        arr = new int[size];  
        Top = -1;  
    }  
}
```

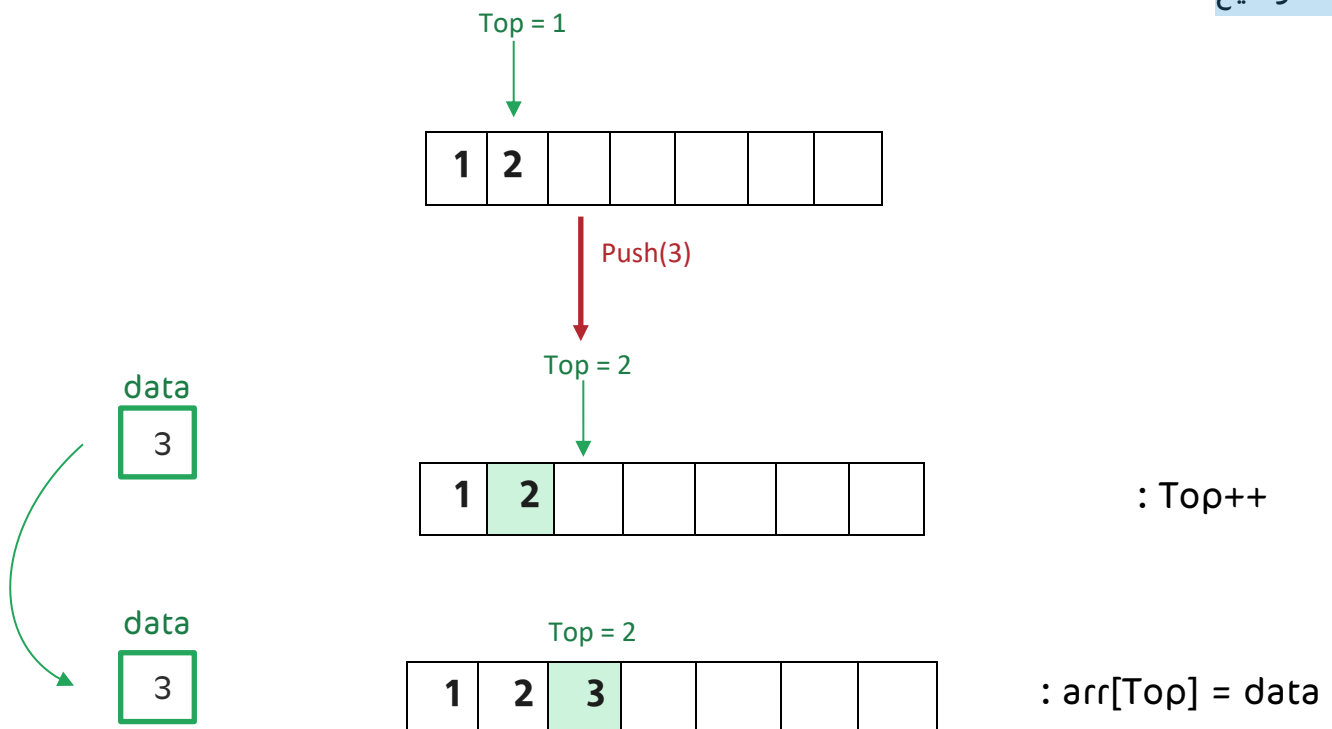


```
void push(int data) {  
    if (Top >= size - 1) {  
        //stack is full  
        System.out.println("Stack Overflow");  
    }else{  
        Top++;  
        arr[Top] = data ;  
    }  
}
```

شرح:

نزيد الـ Top بمقدار 1، ثم نضيف العنصر المطلوب في $arr[Top]$ ونعالج حالة ان تكون الـ arr ممتلئة (أي $Top \geq size - 1$)، فهنا لا يمكننا إضافة أي عنصر إلى arr (أي يحدث stack overflow).

مثال للتوضيح:



```

Int pop() {
    if (Top == -1) {
        // stack is empty
        System.out.println("Stack Underflow");
        return -1;
    } else {
        return arr[Top--];
    }
}

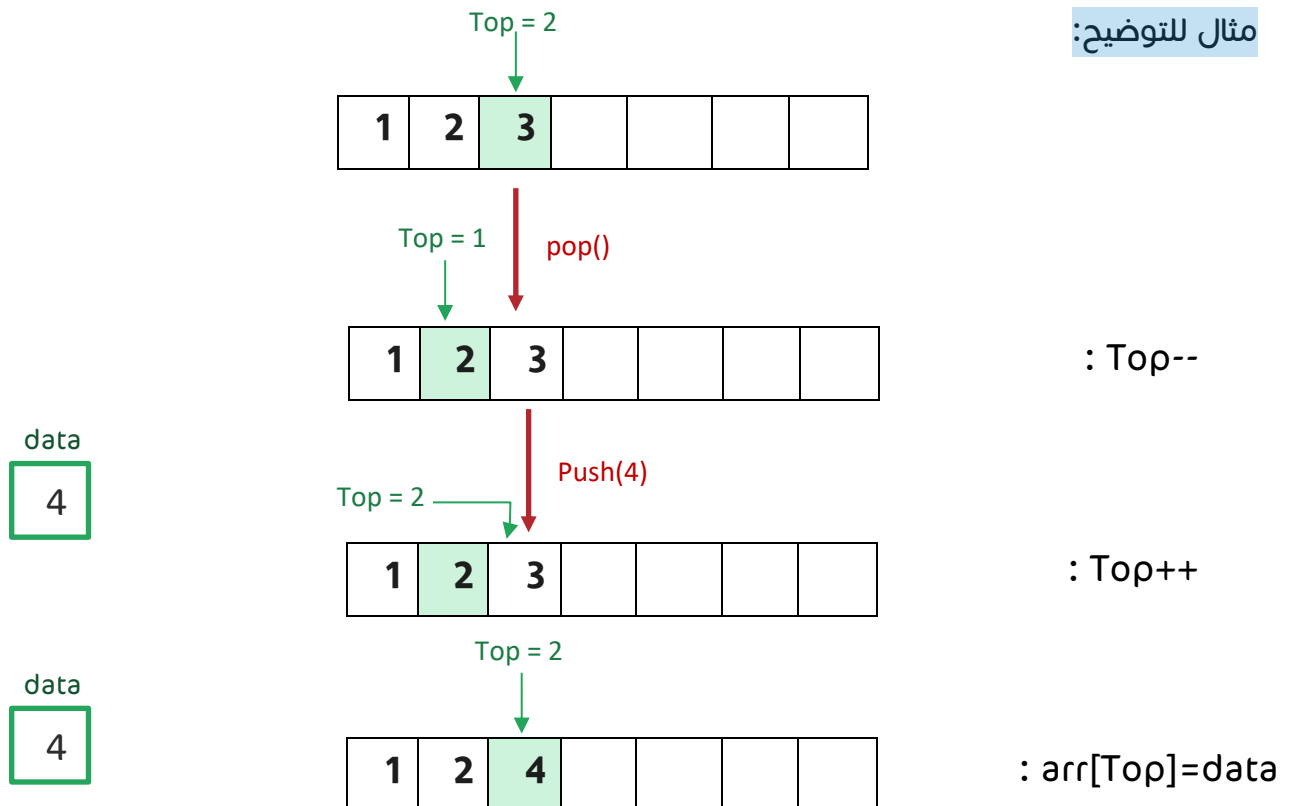
```

شرح:

ننقص الـ Top بمقدار 1، ونعالج حالة أن تكون الـ arr فارغة (أي $Top == -1$) ففي هذه الحالة لا يوجد عناصر حتى نقوم بالحذف.

نلاحظ: أنه لم نقم بحذف العنصر بشكل صريح، حيث فقط قمنا بـ ($Top--$) وهذا كافٍ لأنه عند القيام بـ push لعنصر آخر سيتم تخزينه في $arr[Top+1]$ (أي أنه سيتم الكتابة فوق العنصر القديم).

مثال للتوضيح:



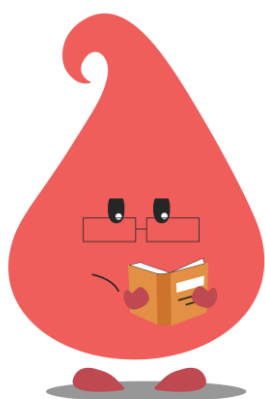
```
int peek() {  
    if (Top == -1) {  
        // stack is empty  
        return -1;  
    }  
    return arr[Top];  
}
```

شرح:

نعيد قيمة العنصر arr[Top] ونعالج حالة إذا كان (Top == -1) فتكون arr فارغة.

```
boolean isEmpty() {  
    return (Top == -1);  
}
```

عند تمثيل stack بمصفوفة سنواجه مشكلة الحجم الثابت للـ stack (fixed size)، لذا ليس من الشائع استخدام array لتمثيل مفهوم stack.



لا تحسبن المجد تمرأ أنت أكله

لا تحسبن المجد تمرأ أنت أكله

حوط بن رئاب الأسدي

تمثيل stack باستخدام Linked List: (وهو الأكثر شيوعاً)

سننشئ class الـ Linked List كما تعلمنا سابقاً، وسيكون الاختلاف هنا في استخدام هذه البنية (أي في التتابع الموجودة فيها والعمليات التي ستجري عليها).

```
public class MyStackL {
    Node head;
    class Node {
        int data;
        Node next;
        Node(int new_data) {
            this.data = new_data;
        }
    }
    void push(int newData) {
        Node newNode = new Node(newData);
        newNode.next = head; head = newNode;
    }
    int pop() {
        if (head == null){
            System.out.println("Stack Underflow");
            return -1;
        }
        else{
            int poppedValue = head.data;
            head = head.next;
            return poppedValue;
        }
    }
    int peek() {
        if (head == null) {
            // stack is empty
            return -1;
        }
        return head.data;
    }
    boolean isEmpty() {
        return head == null;
    }
}
```

نلاحظ أن تابع push() يكافئ تابع insert() (وهو إضافة عنصر في بداية الـ Linked List)

نحذف أول عنصر من الـ Linked List ونعيد قيمته

يعيد الـ data الموجودة في الـ Node التي يشير إليها head.

تمتاز تمثيل الـ stack باستخدام الـ Linked List بالـ dynamic size

تساؤل:

- إذا كانت البنى Array و LinkedList تتيح لنا الوصول إلى أي عنصر داخلها (وليس فقط العنصر الموجود في الأعلى)، فلماذا نقصر وصولنا فقط على آخر عنصر تمت إضافته؟
- لأننا يا أصدقائي نستخدم الـ LinkedList و Array كأدوات لتطبيق مفهوم الـ stack والذي يتبع مبدأ LIFO، ولتحقيق هذا المفهوم نقصر وصولنا على آخر عنصر تمت إضافته فقط.
- أما لو استخدمنا التجول بين العناصر فنكون قد خرجنا عن إطار عمل الـ stack، أي أننا أصبحنا نتعامل مع LinkedList أو Array وليس stack.

ملاحظات عن الـ stack:

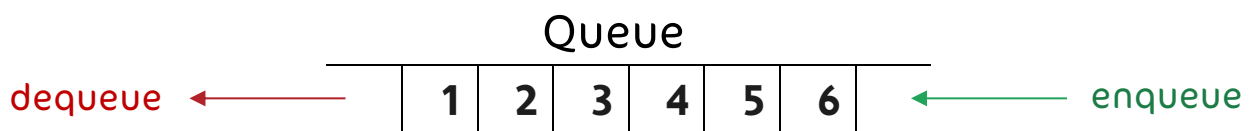
- الإضافة والحذف يتمان بالجهة ذاتها.
- إذا أضفنا في البداية فسنحذف في البداية – كما في الـ LinkedList
- إذا أضفنا في النهاية فسنحذف في النهاية – كما في الـ Array
- احتجنا لـ pointer واحد فقط (يشير إلى آخر قيمة تمت إضافتها)
- تعقيد التوابع : push – pop – peek – isEmpty هو $O(1)$.

Queue

مفهوم الـ Queue (الطابور أو الرتل) :

هو عبارة عن بنية معطيات خطية (linear data structure) يتبع مبدأ FIFO أو (First-In-First-Out) والذي يعني أن أول عنصر تتم إضافته هو أول عنصر يمكننا الوصول إليه (أو حذفه).

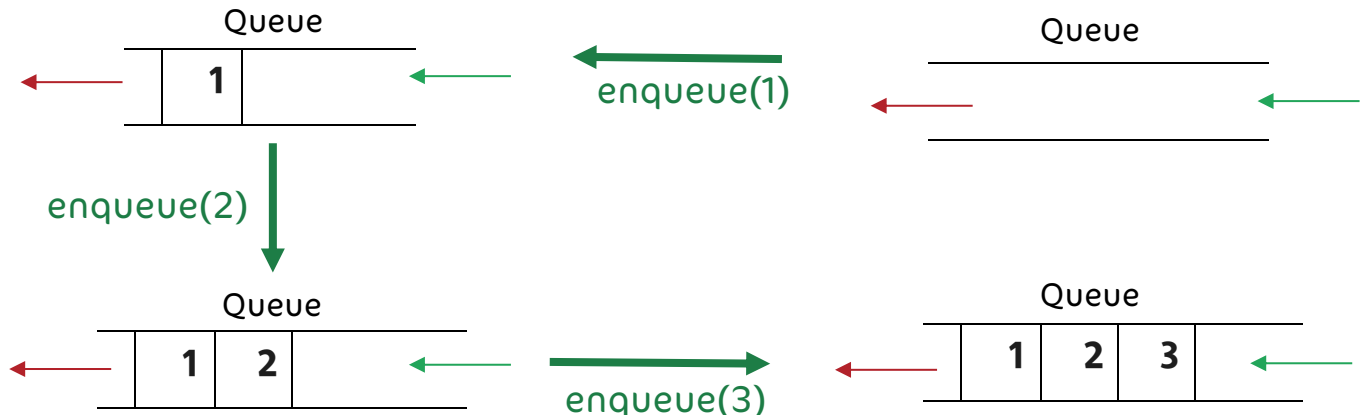
للتوضيح: يمكن تمثيل المعلومات باستخدام الـ Queue كطابور من الناس يقفون لشراء تذاكر، حيث سيتم بيع أول شخص أولاً ثم بيع ثاني شخص وهكذا...



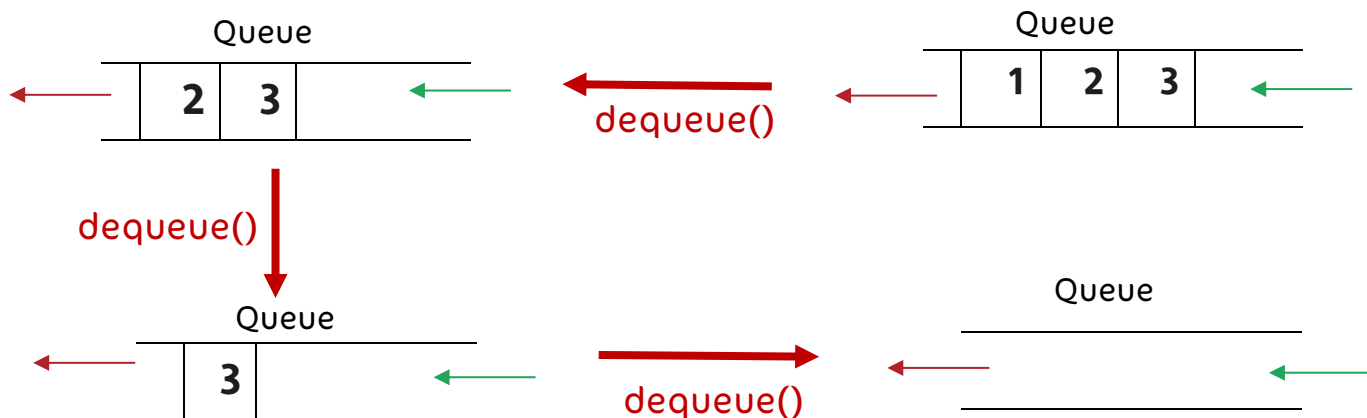
العمليات على الQueue :

1. enqueue(Element e) : "إضافة عنصر في الخلف."

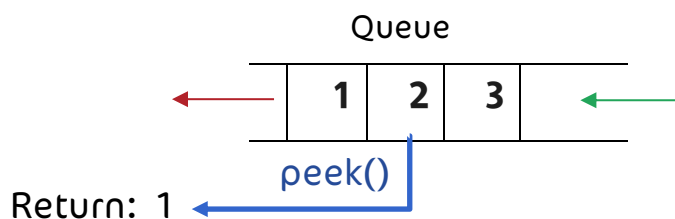
Element هنا ليست data type هي فقط لتوضيح وجود parameter وذلك حسب data type الqueue



2. dequeue() : "حذف العنصر الذي يقع في المقدمة."



3. peek() : "إعادة قيمة العنصر الذي يقع في المقدمة."



4. isEmpty() : "يعيد True إذا كان الqueue فارغاً، و false إذا كان يحوي عنصر أو أكثر."

- Two pointers : إحداهما يشير إلى مقدمة الQueue والآخر يشير إلى نهاية الQueue.
- بنية معطيات خطية : Array أو LinkedList .

فائدة: تطبيق الQueue باستخدام الArray:

```
public class MyQueueA {
    int[] queue;
    int front; // الرأس
    int rear; // الذيل
    int size; // الحجم الحالي
    int capacity; // السعة الكلية
    MyQueueA(int capacity) {
        this.capacity = capacity;
        queue = new int[capacity];
        front = 0;
        rear = -1;
        size = 0;
    }
    void enqueue (int data) {
        if (size == capacity) {
            System.out.println("Queue Overflow");
            return; }
        rear = (rear + 1) % capacity;
        //ال % من أجل العودة لبداية الqueue في حال الوصول لنهايته فيصبح ال rear في بداية ال queue
        //أي كي يكون التنقل دائرياً وذات الشيء لل front
        queue[rear] = data;
        size++;
    }
    int dequeue() {
        if (isEmpty()) {
            System.out.println("Queue Underflow");
            return -1;
        }
        int value = queue[front];
        front = (front + 1) % capacity; // التنقل دائرياً
        size--;
        return value;
    }
}
```

```
int peek() {
    if (isEmpty()) {           // Queue is empty
        return -1; }
    return queue[front];
}

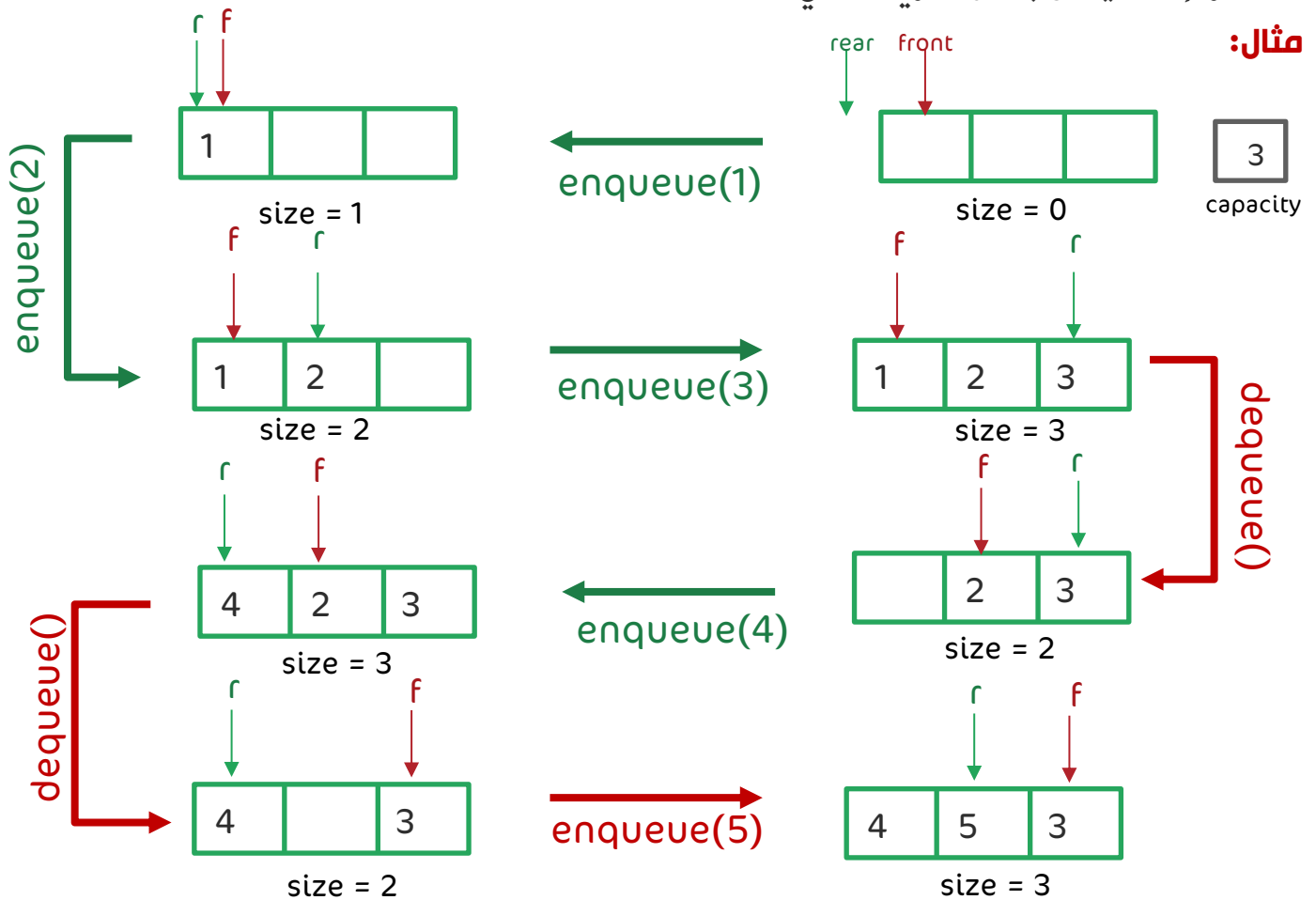
boolean isEmpty() {
    return size == 0; }
}
```

حيث: front: يشير إلى مقدمة الـ Queue ومن عنده يتم الـ peek و الـ dequeue.
 rear: يشير إلى نهاية الـ Queue ومن عنده يتم الـ enqueue.
 size: عدد العناصر الموجودة.
 capacity: حجم المصفوفة.

الحذف والإضافة: (dequeue + enqueue)

الحذف والإضافة يتمان بشكل دائري، كالتالي:

مثال:



- نلاحظ تتغير قيمة الـ front والـ rear بإضافة وحذف العناصر وتتغير مواقع
- و نلاحظ أيضا أنه عند تطبيق الـ queue باستخدام الـ array ستواجهنا مشكلة الـ fixed size (الحجم الثابت).

تطبيق الQueue باستخدام الLinkedList :

ستكون بنية class الLinkedList نفسها، لكن الاختلاف سيكون باستخدام هذه البنية (أي بالعمليات عليها والـ methods فيها).

```
public class MyQueueL {
    Node front, rear;
    class Node {
        int data;
        Node next;
        Node(int data) {
            this.data = data;
        }
    }
}
```

الـ rear: يكافئ الـ tail، حيث يشير إلى نهاية الLinkedList (والتي سنعتبرها نهاية الطابور) سيتم الenqueue من جهته "يشير إلى آخر عنصر تمت إضافته".

الـ front: يكافئ الـ head، حيث يشير إلى بداية الLinkedList (والتي سنعتبرها بداية الطابور) سيتم الDequeue والـ peek من جهته "يشير إلى أول عنصر تمت إضافته".

1. تابع enqueue:

شرح:

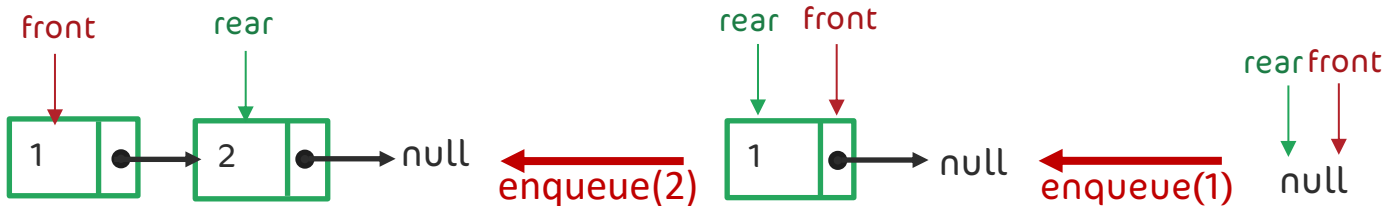
هنا ناقشنا حالتين:

(1) أن تكون الLinkedList فارغة

← نجعل الـ front و الـ rear يشيران إلى العنصر المضاف

(2) ألا تكون الLinkedList فارغة

← نجعل Node من جهة الـ rear.

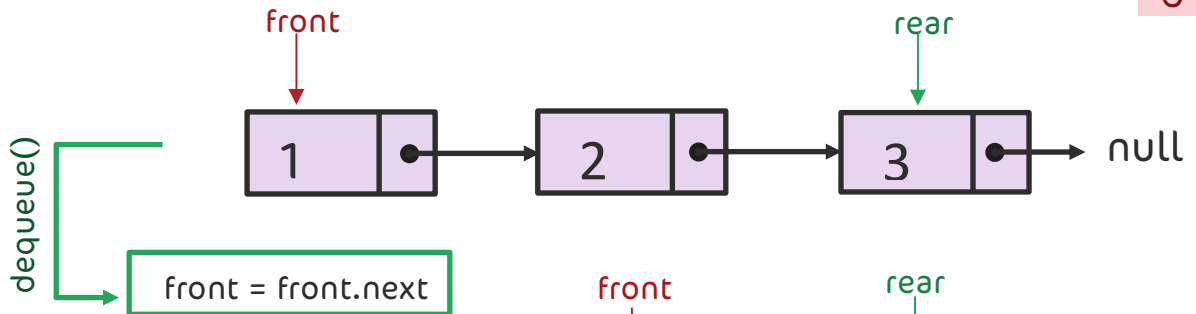
مثال:

```
void dequeue() {
    if (front == null)
        System.out.println("Queue Underflow");
    else
        front = front.next;
    if (front == null) {
        rear = null;
    }
}
```

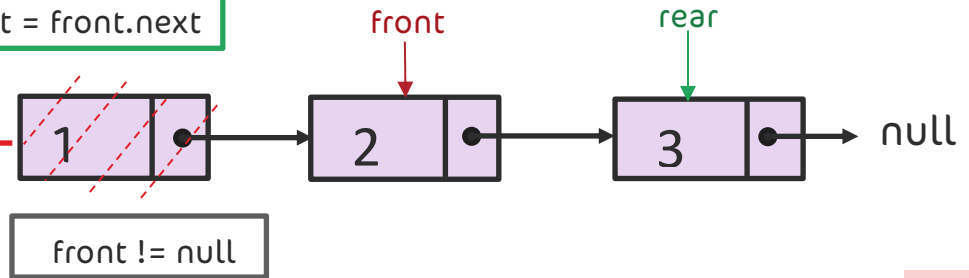
شرح:

- (1) إذا كانت الـ LinkedList فارغة
← لا يمكن أن نحذف شيء.
- (2) إذا لم تكن الـ LinkedList فارغة
← نحذف العنصر الذي تشير إليه الـ front.

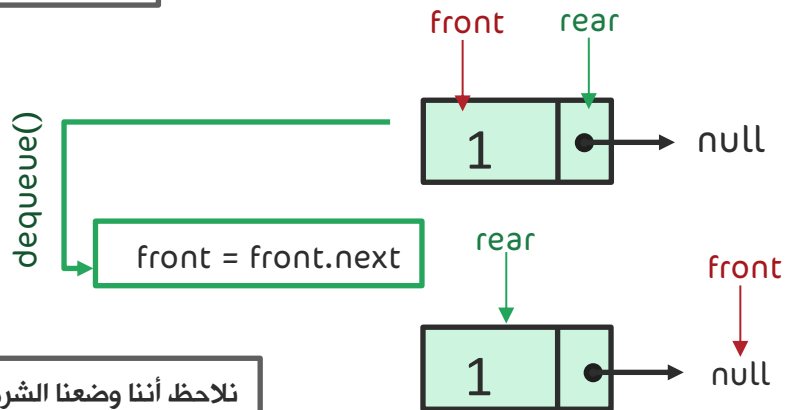
مثال 1:



سيقوم الـ garbage collector بحذف العنصر لأنه لا يوجد ما يؤشر عليه

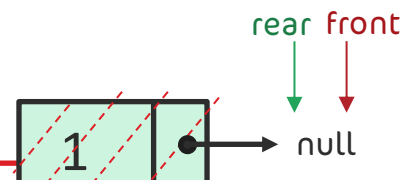


مثال 2:



front == null → rear = null

نلاحظ أننا وضعنا الشرط: if(front==null){ rear =null;} والذي يعني ان الـ LinkedList مكونة من عنصر واحد فقط، فعند جعل front = front.next ← سيصبح الـ front = null لكن يجب جعل الـ rear يشير إلى null أيضاً حتى تحذف العقدة وهذا ما يحققه الشرط.



3. تابع peek():

شرح:

```
int peek() {
    if (front != null) {
        return front.data;
    }
    return -1;
}
```

إذا لم تكن الـ LinkedList فارغة
 ← يعيد الـ data في الـ Node التي يشير إليها الـ front.
 وإلا ← يعيد (-1).

4. تابع isEmpty():

شرح:

```
boolean isEmpty() {
    return front == null;
}
```

إذا كانت الـ front == null ← الـ linkedList فارغة ← نعيد true.
 وإلا ← نعيد false.

ملاحظات عن الـ Queue:

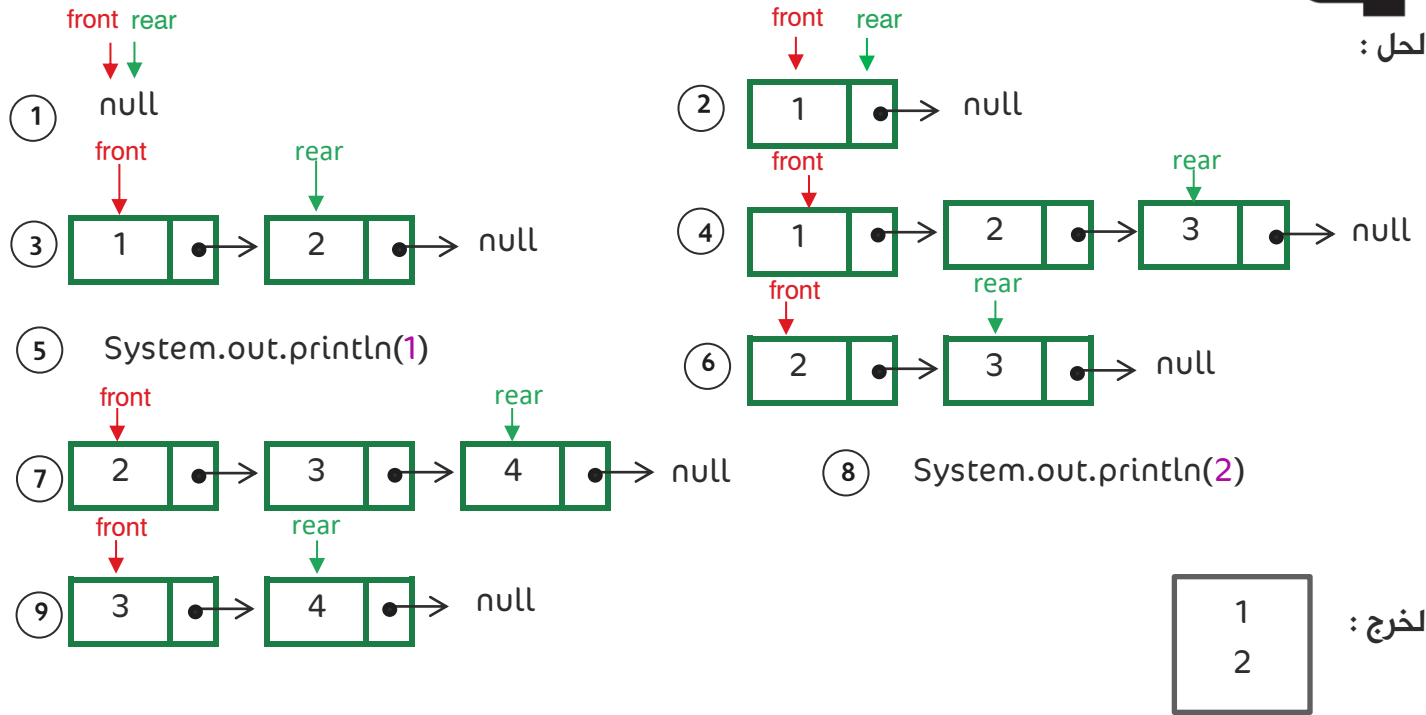
- الإضافة والحذف يتمّان باتجاهين متعاكسين (إذا أضفنا من النهاية فسنحذف من البداية – كما في الـ LinkedList)
- احتجنا إلى 2 pointers، أحدهما يشير إلى أول عنصر تمّت إضافته (ومن عنده يتم حذف العناصر)، والآخر يشير إلى آخر عنصر تمّت إضافته (ومن عنده يتم إضافة العناصر).
- تعقيد التوابع : enqueue - dequeue - peek - isEmpty هو $O(1)$.

تمرين 1 : ما هو خرج الكود التالي :

```
public class ex1 {
    public static void main(String[] args) {
        MyQueueL q = new MyQueueL();
        1. q.enqueue(1);
        2. q.enqueue(2);
        3. q.enqueue(3);
        4. System.out.println(q.peek());
        5. q.dequeue();
        6. q.enqueue(4);
        7. System.out.println(q.peek());
        8. q.dequeue();
    }}

```

استخدمنا كلاس MyQueue الذي
 شرحناه سابقاً .



تمرين 2 : كتابة تابع لقلب كلمة باستخدام الـ stack .

1. أنشأنا stack عناصرها من نوع

Character

```
public static String reverseString(String input) {
1. Stack<Character> stack = new Stack<>();
2. for (char ch : input.toCharArray()) {
3.     stack.push(ch);
4. }
5. String reversed = "";
6. while (!stack.isEmpty()) {
7.     reversed += stack.pop();
8. }
9. return reversed;
}
```

■ ملاحظة :

لقد استخدمنا كلاس الـ stack الجاهز في لغة Java هنا

2. حلقة for تمر على كل أحرف الـ String

■ ملاحظة :

التابع toCharArray() يقوم بتحويل الـ string إلى مصفوفة من char

3. نضيف كل حرف إلى الـ stack .

5. أنشأنا متغير string فارغ (وهو المتغير الذي سنخزن به الحل أو الكلمة بعد القلب) .

6. حلقة while تبقى مستمرة طالما أن الـ stack ليست فارغة .

7. تقوم بـ pop لكل عنصر من الـ stack ونضيفه إلى الـ string الذي نخزن به الحل (الكلمة المقلوبة).

للتذكرة :

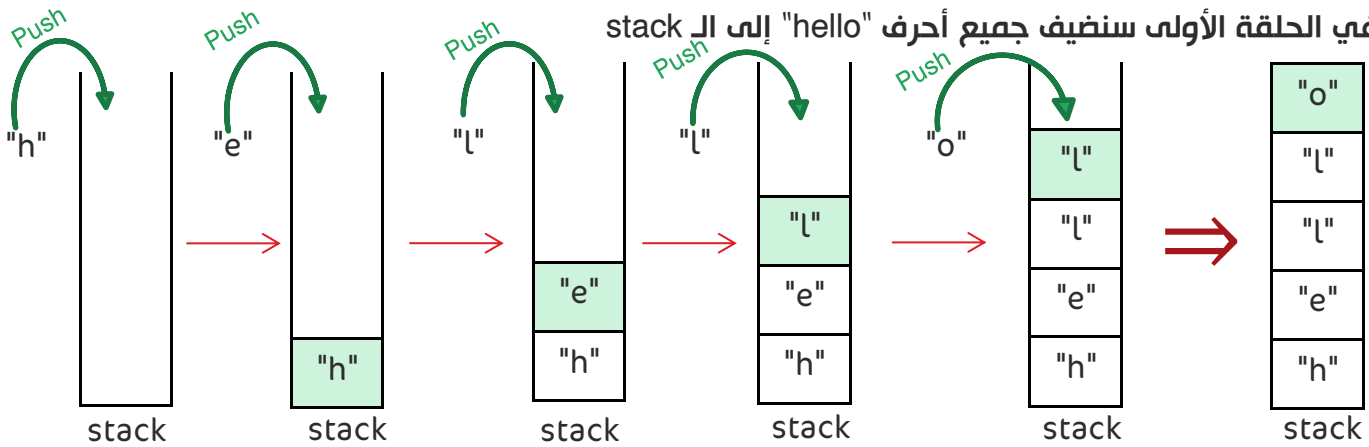
تابع الـ pop الموجود في الـ stack الجاهزة في الـ Java يقوم بإعادة آخر عنصر في الـ stack وحذفه

9. نرجع reversed (وهو الـ string الذي خزن الكلمة المقلوبة)

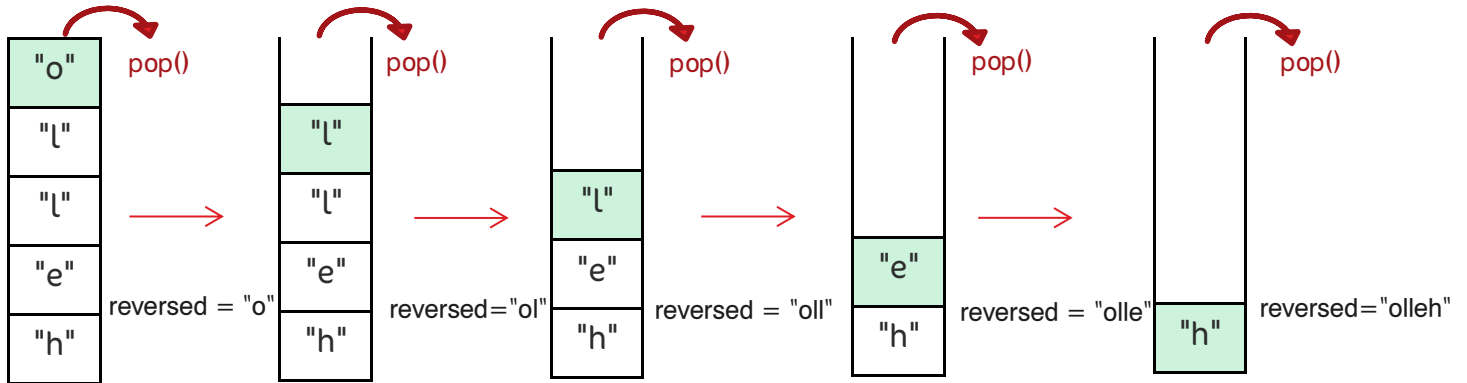
مثال للتوضيح :

بفرض كتبنا `system.out.println(reversestring "hello");`

■ في الحلقة الأولى سنضيف جميع أحرف "hello" إلى الـ stack



■ في الحلقة الثانية ستقوم بـ pop لكل عنصر نضيفه إلى reversed



← سوف يطبع "olleh"

تمرين 3 : اكتب تابع للتحقق من أن الأقواس متوازنة في العبارة المعطاة .

■ الأقواس متوازنة : أي أنه لكل قوس مفتوح قوس يغلقه

مثال : "[({})]" : عبارة غير متوازنة

مثال : "[({({}))}]": عبارة متوازنة

```
import java.util.Stack;
public class ex3 {
    private boolean isOpen(char ch) {
        return ch == '(' || ch == '{' || ch == '[';
    }
    private boolean match(char open, char close) {
        return (open == '(' && close == ')') ||
            (open == '{' && close == '}') ||
            (open == '[' && close == ']');
    }
    public boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();
        for (char ch : s.toCharArray()) {
            if (isOpen(ch)) {
                stack.push(ch);
            } else {
                if (stack.isEmpty() || !match(stack.pop(), ch)) {
                    return false;
                }
            }
        }
        return stack.isEmpty();
    }
}
```

يوجد لدينا تابعان مساعدان هما :

isOpen () : يتحقق إذا كان القوس مفتوح أو لا .

match () : يتحقق إذا كان القوسان الممرين له متوافقان ، أي : "(", ")" مثلاً

• التابع isValid () هو حل

مسألتنا ، يعمل كالتالي :

نمر على عناصر العبارة المدخلة ، وعند كل عنصر :

▪ إذا كان قوس مفتوح نضيفه إلى stack

▪ إذا كان قوس مغلق يوجد

حالتين:

(1) إذا كان يحقق ال match method مع آخر عنصر تمت إضافته إلى ال stack إذاً نكمل الحلقة (لا

نضيف القوس المغلق) ونحذف آخر عنصر الذي هو القوس المفتوح المتوافق معه .

(2) إذا لم يكن يحقق ال match method مع آخر عنصر تمت إضافته إلى ال stack أو كانت ال stack

فارغة (أي أنه لا يوجد أقواس مفتوحة قبل هذا القوس المغلق) إذاً سيعيد false (تكون العبارة ليست متوازنة)

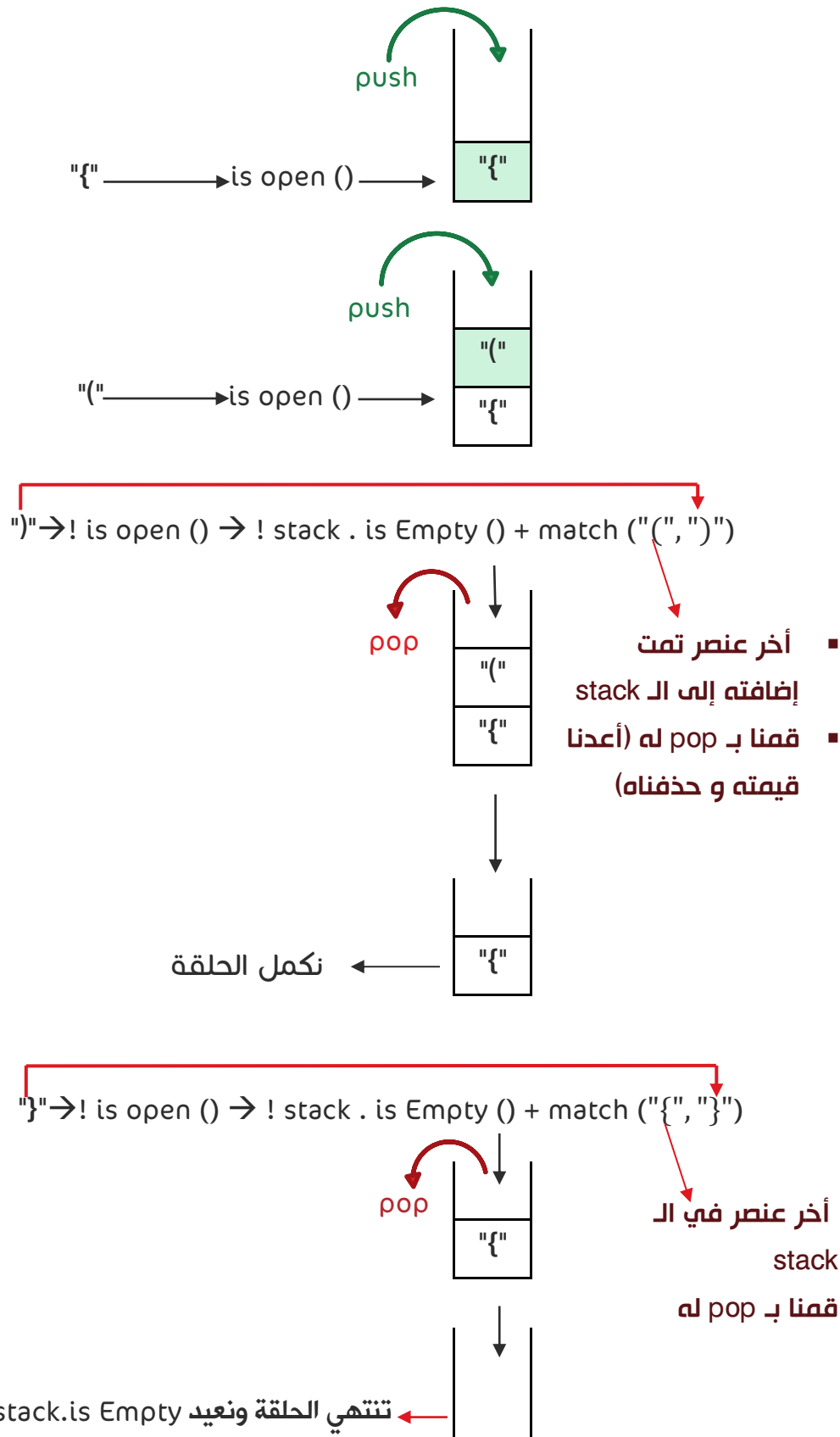
• إذا أكملنا تنفيذ الحلقة (أي أن كل قوس مغلق في العبارة كان متوافق مع آخر عنصر في ال stack والذي سيكون

قوس مفتوح) وبالتالي في نهاية التابع ستكون ال stack فارغة .

مثال للتوضيح :

بفرض مررنا : "{}()"

سنمر على عناصر العبارة:



تمرين 4 : كتابه تابع يقوم بالتالي:

من أجل مصفوفة معطاة ، يجد العنصر الأكبر التالي (The Next Greater Element(NGE)) لكل عنصر

■ ال NGE لعنصر في المصفوفة: هو أول عنصر يقع بعده و أكبر منه .

■ وإذا لم يكن للعنصر NGE ، فيكون $NGE = -1$ لهذا العنصر .

■ بداية سنحل المسألة بطريقة عادية (دون استخدام Stack أو Queue)

⚡ شرح :

الكود عبارته عن حلقة for تقارن كل عنصر $a[i]$ بجميع

العناصر التي تليه في المصفوفة ، ولدينا أن next يأخذ

-1 عند كل عنصر:

■ إذا أوجدنا NGE لهذا العنصر تتغير قيمه next

وتأخذ ال NGE ، ثم نكسر الحلقة ونطبع قيمة

next .

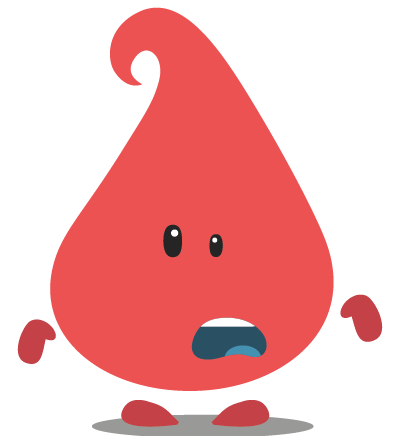
■ إذا لم نجد NGE لهذا العنصر ستبقى قيمة

next هي -1 ، وعند انتهاء ال for الداخلية لهذا

العنصر نطبع ال next (نطبع -1)

⚡ التعقيد :

$$T(n) = \frac{n(n+1)}{2} = O(n^2)$$



شرح:

سنمر على عناصر المصفوفة بدءاً بآخر عنصر

وصولاً إلى أول عنصر ، وعند كل عنصر ستقوم بـ :

1. مقارنة مع العناصر الموجودة في الـ stack

(والتي ستكون قبله في المصفوفة لأننا

نمر على العناصر بالعكس)

المقارنة مع العناصر تتم كالتالي :

نقارن العنصر (وليكن a) مع آخر عنصر

موجود في الـ stack (وليكن b):

■ إذا كان b يحقق أنه NGE للعنصر a

:وجدنا الـ NGE للعنصر a وهو b .

■ إذا لم يكن b عبارة عن NGE لـ a :

نقوم بـ pop لـ b ونكرر المقارنة

حتى تنتهي عناصر الـ stack أو حتى

نجد NGE للعنصر a

■ إذا وجدنا NGE نضع قيمته في الـ res .

■ إذا انتهت عناصر الـ stack ولم نجد NGE

للعنصر نضع -1 في الـ res .

2. إضافته إلى الـ stack "st"

```
import java.util.Stack;

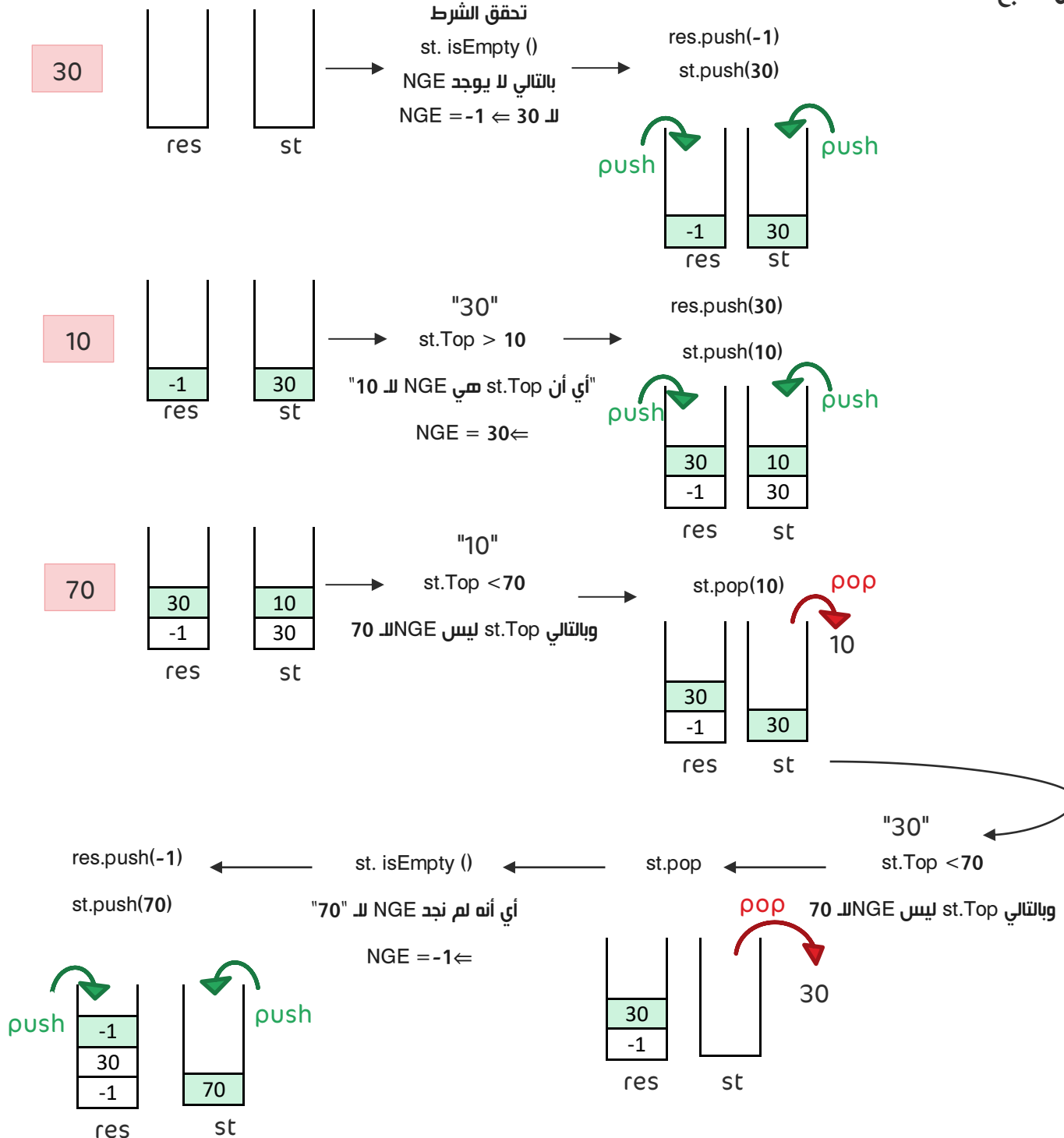
public class ex4{
    void NGE(int arr[], int n) {
        // stack "st" يساعدنا في إيجاد الـ NGE حيث يحفظ العنصر الحالي
        // من المصفوفة التي نمر عليها لأننا سنحتاجه لإيجاد NGE العنصر
        // التالي
        Stack<Integer> st = new Stack<>();
        // Stack "res" يحفظ NGE العناصر
        Stack<Integer> res = new Stack<>();
        for (int i = n - 1; i >= 0; i--) {
            // حلقة while طالما أن الـ "st" ليس فارغاً والعنصر الأخير فيه
            // أصغر من العنصر الحالي في المصفوفة فيقوم بحذف العناصر من
            // الـ "st" stack ويستمر بذلك حتى يصبح العنصر الأخير في st أكبر
            // أو يساوي العنصر الحالي
            while (!st.isEmpty() && st.peek() < arr[i]){
                st.pop();
            }
            // بعد تنفيذ حلقة while إذا أصبح st فارغاً هذا يعني أننا مررنا على
            // جميع عناصر "st" دون أن نجد الـ NGE فنضيف -1 على الـ stack "res"
            if (st.isEmpty())
                res.push(-1);
            // في حال لم يكن فارغاً هذا يعني أننا وجدنا NGE للعنصر الحالي وهو
            // آخر عنصر في الـ "st" stack فنضيفه على الـ res
            else
                res.push(st.peek());
            // نضيف العنصر الحالي في المصفوفة على الـ "st" stack
            st.push(arr[i]);
        }

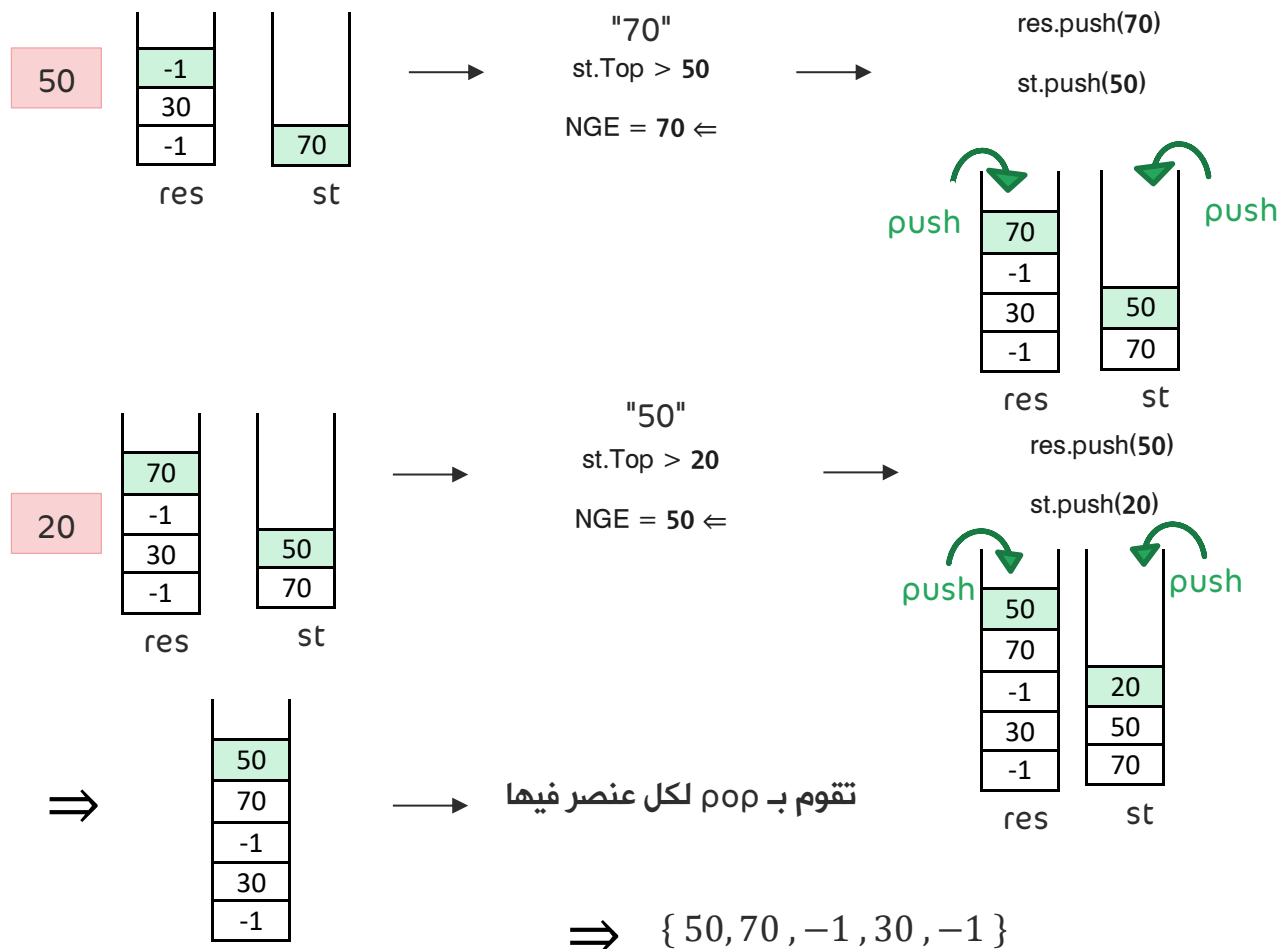
        while (!res.isEmpty())
            System.out.println(res.pop());
    }
}
```

مثال: لتكن لدينا المصفوفة : { 20 , 50 , 70 , 10 , 30 }

الناتج سيكون : { 50, 70 , -1 , 30 , -1 }

آلية عمل التابع :





تلخيص :

Stack	Queue	
LIFO	FIFO	تتبع مبدأ
نحتاج إلى Pointer واحد فقط Top : يشير إلى آخر عنصر تمت إضافته	نحتاج إلى 2 Pointers front : يشير إلى أول عنصر تمت إضافته rear : يشير إلى آخر عنصر تمت إضافته	عدد ال Pointers
يتمان بجهة واحدة (بجهة ال Top) .	يتمان بجهتين : - الإضافة تتم من جهة rear -الحذف يتم من جهة front	الإضافة والحذف
$O(1)$	$O(1)$	تعقيد العمليات عليها

THE END